

Trabalho Prático 2: Modelagem de Redes Sociais Acadêmicas

Bento Gabriel Araújo Matos

Matrícula: 2025049603

Departamento de Ciência da Computação
Universidade Federal de Minas Gerais (UFMG)
Belo Horizonte – MG – Brasil

bentogabrielmatos@gmail.com

1. Introdução

Este trabalho aborda o desenvolvimento de um sistema para modelagem de redes sociais acadêmicas para a plataforma ConectaDCC. O sistema deve manter dinamicamente dois grafos distintos: um grafo social direcionado, responsável por representar as relações de seguimento entre usuários, e um grafo bipartido não direcionado, utilizado para representar os interesses dos usuários em diferentes temas acadêmicos e pessoais. Além disso, o sistema deve permitir operações de inserção e remoção de relações, consultas sobre conexões e interesses, cálculo de popularidade de temas e alteração dinâmica da representação interna dos grafos durante a execução do programa.

Dessa forma, o principal objetivo deste trabalho é propor e implementar estruturas de dados e algoritmos eficientes capazes de representar os grafos e realizar todas as operações especificadas com baixo custo computacional. Para isso, foi desenvolvido um programa em C++11 utilizando Tipos Abstratos de Dados (TADs) para modularizar os componentes do sistema, incluindo usuários, temas, grafos e estruturas auxiliares de mapeamento. O projeto também contempla o suporte a diferentes formas de armazenamento dos grafos, permitindo alternar dinamicamente entre listas de adjacência, matrizes de adjacência e CSR (Compressed Sparse Row) sem perda das informações já armazenadas.

Além da implementação funcional do sistema, o trabalho também busca analisar experimentalmente o desempenho das diferentes representações dos grafos em cenários variados de utilização, considerando fatores como densidade das conexões, frequência de consultas e quantidade de atualizações realizadas. A partir dessa análise, pretende-se identificar os impactos das decisões de implementação sobre o custo temporal e espacial das operações.

Com isso, na Seção 2 serão descritas as estruturas de dados, os Tipos Abstratos de Dados e os algoritmos utilizados na implementação do sistema. Na Seção 3 serão apresentadas as análises de complexidade de tempo e espaço das operações implementadas. Na Seção 4 serão discutidas as estratégias de robustez e programação defensiva empregadas no desenvolvimento. Na Seção 5 serão apresentados os experimentos realizados e a análise dos resultados obtidos. Por fim, na Seção 6 serão apresentadas as conclusões e os principais aprendizados obtidos ao longo do desenvolvimento deste trabalho.

2. Método

O sistema foi desenvolvido em C++11 utilizando uma arquitetura modular baseada em Tipos Abstratos de Dados (TADs), separando as responsabilidades de armazenamento, representação e manipulação das informações. A implementação mantém dois grafos distintos: um grafo social direcionado, responsável pelas relações de seguimento entre usuários, e um grafo bipartido não

direcionado, utilizado para representar os interesses dos usuários em temas acadêmicos e pessoais.

A execução do programa ocorre por meio da leitura sequencial dos comandos fornecidos na entrada. Cada comando é interpretado e encaminhado para os métodos responsáveis pela operação correspondente, realizando atualizações ou consultas sobre os grafos e imprimindo os resultados no formato especificado.

2.1. Estruturas de Dados

- **Listas de Adjacência:** Utilizadas como uma das representações possíveis dos grafos. Cada vértice mantém uma lista contendo apenas seus vértices adjacentes, tornando essa abordagem mais eficiente em grafos esparsos, devido ao menor consumo de memória e ao custo reduzido para inserção de arestas.
- **Matriz de Adjacência:** Representação alternativa baseada em uma matriz booleana bidimensional, na qual cada posição indica a existência ou não de uma aresta entre dois vértices. Essa abordagem permite consultas de adjacência em tempo constante, sendo mais adequada para grafos densos.
- **CSR (Compressed Sparse Row):** Estrutura adicional implementada como ponto extra do trabalho. Nessa representação, as arestas são armazenadas de forma compacta em vetores lineares, reduzindo significativamente o consumo de memória em grafos esparsos quando comparado à matriz de adjacência.
- **Vetores Dinâmicos:** Utilizados pelo TAD Dicionário para armazenar usuários, temas e os mapeamentos entre identificadores lógicos e vértices dos grafos. Os vetores possuem expansão dinâmica para suportar o crescimento do sistema durante a execução.

2.2. Tipos Abstratos de Dados

- **Usuário:** TAD responsável por armazenar as informações associadas a cada usuário do sistema, incluindo identificador, nome e idade.
- **Tema:** Representa um tema acadêmico ou pessoal, armazenando identificador, nome e tipo.
- **Grafo:** Principal TAD do sistema, responsável pela representação dos grafos social e de temas. O TAD suporta grafos direcionados e não direcionados, além de permitir a troca dinâmica da representação interna entre listas de adjacência, matriz de adjacência e CSR sem perda de vértices ou arestas. Também encapsula as operações de inserção, remoção e consulta de arestas.
- **Dicionário:** Estrutura auxiliar responsável por relacionar os identificadores de usuários e temas aos respectivos objetos e vértices utilizados nos grafos. Esse TAD funciona como uma camada de mapeamento, permitindo acesso eficiente às estruturas armazenadas.

2.3. Algoritmos

Os algoritmos implementados foram desenvolvidos para realizar as operações especificadas no enunciado de maneira eficiente, considerando as diferentes representações possíveis dos grafos.

- **Inserção de Usuários e Temas:** Ao cadastrar um novo usuário ou tema, o sistema cria o objeto correspondente, adiciona os vértices necessários nos grafos e registra todas as associações no TAD Dicionário.
- **Inserção e Remoção de Arestas:** As operações de seguir e deixar de seguir usuários são implementadas por meio da adição e remoção de arestas direcionadas no grafo social. Já os interesses dos usuários em temas são representados por arestas no grafo bipartido.

- **Consultas de Adjacência:** Utilizadas para verificar relações entre usuários ou entre usuários e temas. Dependendo da representação utilizada, as consultas podem ser realizadas por busca em listas ou por acesso direto à matriz.
- **Listagem de Relacionamentos:** As operações de listagem percorrem os vértices adjacentes para obter seguidores, usuários seguidos, amigos e temas associados. No caso da consulta de amigos, o sistema verifica relações mútuas de seguimento entre usuários.
- **Consulta de Popularidade:** A popularidade de um tema é obtida pela quantidade de usuários conectados ao vértice correspondente no grafo de temas.
- **Migração de Representação:** Quando o modo de armazenamento é alterado, o sistema percorre a estrutura atual do grafo e reconstrói suas arestas na nova representação selecionada, preservando integralmente o estado lógico do sistema.

2.4. Configurações Utilizadas para Testes

As seguintes especificações foram utilizadas no desenvolvimento e testes do programa:

- **Sistema Operacional:** Ubuntu 24.04.4 LTS
- **Linguagem:** C++11
- **Compilador:** gcc 13.3.0
- **Processador:** AMD Ryzen 5 3400G 3.70 GHz
- **RAM disponível:** 8 GB

3. Análise de Complexidade

A análise de complexidade do sistema considera os custos de tempo e espaço das principais operações implementadas. Para a análise, considera-se:

- U : número de usuários;
- T : número de temas;
- V : número de vértices do grafo;
- E : número de arestas do grafo;
- $\deg(v)$: grau do vértice v .

Como o sistema suporta múltiplas representações para os grafos, a complexidade das operações varia de acordo com o modo de armazenamento utilizado.

3.1. Complexidade de Tempo

3.1.1 Operações do TAD Dicionário

O TAD Dicionário utiliza vetores indexados diretamente pelos identificadores dos elementos, permitindo acesso em tempo constante.

- **Busca de usuários, temas e vértices:** Como os identificadores são utilizados diretamente como índices nos vetores, operações de consulta possuem complexidade $O(1)$.
- **Inserção de usuários e temas:** Em condições normais, a inserção ocorre em $O(1)$ amortizado. Entretanto, quando os vetores atingem o máximo, é necessário expandi-los.
- **Expansão dos vetores:** Durante a realocação, todos os elementos armazenados precisam ser copiados para novos vetores maiores, resultando em complexidade $O(T + U)$.

3.1.2 Operações do TAD Grafo

As operações do TAD Grafo possuem comportamentos distintos dependendo da representação utilizada.

Listas de Adjacência Nas listas de adjacência, cada vértice armazena apenas seus vizinhos diretos.

- **Inserção de arestas:** A inserção exige percorrer os vizinhos do vértice para manter a lista ordenada, resultando em complexidade $O(deg(v))$.
- **Remoção de arestas:** É necessário localizar a aresta desejada na lista do vértice, resultando em complexidade $O(deg(v))$.
- **Consulta de adjacência:** A verificação da existência de uma aresta exige busca na lista de vizinhos, possuindo custo $O(deg(v))$.
- **Listagem de vizinhos:** Como os vizinhos já estão armazenados diretamente na lista, o acesso à estrutura possui custo $O(1)$, enquanto a iteração sobre os elementos é $O(deg(v))$. Quando é necessário obter os amigos mútuos em grafos direcionados, o sistema realiza a interseção por intercalação de duas listas ordenadas (saída e entrada), com complexidade $O(deg_{out}(v) + deg_{in}(v))$.

Essa representação é mais eficiente em grafos esparsos, pois armazena apenas as arestas existentes.

Matriz de Adjacência Na matriz de adjacência, cada posição da matriz indica diretamente a existência ou não de uma aresta entre dois vértices.

- **Inserção e remoção de arestas:** Ambas as operações possuem custo $O(1)$, pois basta alterar o valor de uma posição da matriz.
- **Consulta de adjacência:** Também ocorre em $O(1)$, já que o acesso à posição da matriz é direto.
- **Listagem de vizinhos:** Para obter todos os vizinhos de um vértice, é necessário percorrer toda a linha ou coluna correspondente na matriz, resultando em custo $O(V)$.

Apesar do excelente desempenho para consultas de adjacência, essa abordagem apresenta alto custo de memória em grafos grandes.

CSR (Compressed Sparse Row) A representação CSR foi implementada como estrutura adicional para grafos esparsos.

- **Inserção e remoção de arestas:** Como a estrutura CSR não é adequada para modificações dinâmicas, o sistema converte temporariamente o grafo para listas de adjacência, realiza a operação e reconstrói a estrutura CSR. Dessa forma, essas operações possuem custo elevado, chegando a $O(V + E)$ por operação, dominado pelas duas conversões completas entre representações.
- **Consulta de adjacência:** O acesso aos vizinhos de um vértice ocorre em tempo proporcional ao seu grau, resultando em $O(deg(v))$.
- **Listagem de vizinhos:** Pode ser realizada em $O(1)$ para localizar o intervalo correspondente ao vértice, mas quando é necessário obter vizinhos de chegada e, consequentemente, a interseção é necessário verificar todas as linhas, resultando em $O(V + E)$.

3.1.3 Troca de Representação dos Grafos

A operação de troca de modo percorre os vértices e arestas do grafo atual para reconstruir a nova representação interna.

- **Conversões envolvendo matriz e lista:** É necessário percorrer toda a matriz para fazer a conversão e manter ordem relativa, além disso é necessário percorrer as listas de adjacência em ambos sentidos, resultando em $O(V^2 + E)$.
- **Conversões envolvendo lista e CSR:** É necessário alocar os vetores e percorrer linearmente as estruturas de origem, resultando em $O(V + E)$.
- **Conversões envolvendo matriz e CSR:** Conversão similar a matriz/lista, também resulta em $O(V^2 + E)$.

3.2. Complexidade de Espaço

O consumo de memória também depende diretamente da representação adotada pelos grafos.

- **Listas de Adjacência:** Necessitam armazenar apenas os vértices e arestas existentes, resultando em complexidade espacial $O(V + E)$.
- **Matriz de Adjacência:** Como todas as possíveis conexões entre vértices são armazenadas, o consumo de memória é $O(V^2)$, independentemente da quantidade real de arestas.
- **CSR:** Armazena apenas as arestas existentes e vetores auxiliares de indexação, possuindo custo $O(V + E)$, apesar de mesma ordem assintótica, é consistentemente menor que em listas.
- **TAD Dicionário:** Os vetores de mapeamento e armazenamento de objetos consomem $O(U + T)$.
- **Buffers auxiliares:** Algumas operações utilizam vetores temporários para armazenamento intermediário de adjacências e conversões entre representações, adicionando custo auxiliar proporcional a $O(V)$.

4. Estratégias de Robustez

O sistema incorpora mecanismos de programação defensiva com o objetivo de garantir consistência dos dados, evitar falhas durante a execução e facilitar a manutenção do código. Essas estratégias envolvem validação de entradas, tratamento de exceções e gerenciamento seguro de memória.

4.1. Validação das Entradas

Todas as linhas da entrada são analisadas antes da execução das operações. Inicialmente, o sistema verifica se o comando informado pertence ao conjunto de operações válidas especificadas no enunciado.

Após a identificação do comando, os parâmetros são extraídos e validados individualmente. São realizadas verificações relacionadas à quantidade correta de argumentos, tipos esperados e presença de parâmetros excedentes na linha. Dessa forma, evita-se que comandos mal formatados sejam processados incorretamente.

Também foram implementadas validações específicas para os modos de armazenamento dos grafos, garantindo que apenas os modos suportados pelo sistema possam ser utilizados, bem como validações para os tipos de temas cadastrados.

4.2. Validação de Identificadores

Antes da realização de consultas ou modificações nos grafos, o sistema verifica se os identificadores de usuários e temas referenciados existem no estado atual da aplicação. Essas verificações impedem acessos inválidos às estruturas internas e evitam comportamentos indefinidos decorrentes de referências inexistentes.

Além disso, funções auxiliares garantem que apenas valores inteiros não negativos sejam aceitos para identificadores e demais parâmetros numéricos.

4.3. Tratamento de Exceções

A implementação utiliza blocos `try/catch` para capturar erros de execução de forma controlada. Exceções são utilizadas para sinalizar problemas relacionados ao processamento dos comandos e à validade dos parâmetros informados.

Com isso, falhas localizadas podem ser tratadas sem comprometer completamente a execução do programa, facilitando a identificação de erros e aumentando a robustez da aplicação.

4.4. Gerenciamento de Memória

Como o sistema utiliza alocação dinâmica de memória para armazenar grafos, vetores e objetos do domínio, foram implementados destrutores responsáveis pela liberação adequada de todos os recursos utilizados.

Além disso, as estruturas dinâmicas realizam expansão controlada da capacidade de armazenamento quando necessário, preservando os dados previamente armazenados durante as realocações. Esse mecanismo permite que o sistema cresça dinamicamente sem comprometer a integridade das informações.

Também foram utilizadas verificações sobre ponteiros e estruturas internas antes da realização de operações de destruição ou acesso, reduzindo o risco de falhas relacionadas à memória.

4.5. Controle de Consistência

O sistema mantém a consistência entre os grafos e o TAD Dicionário durante toda a execução. Sempre que novos usuários ou temas são cadastrados, seus respectivos vértices e associações são registrados simultaneamente nas estruturas necessárias.

Da mesma forma, durante a troca da representação interna dos grafos, o estado lógico do sistema é preservado integralmente, garantindo que nenhuma aresta ou vértice seja perdido durante o processo de conversão.

5. Análise Experimental

A análise experimental teve como objetivo avaliar o desempenho das diferentes representações de grafos implementadas no sistema ConectaDCC: listas de adjacência, matriz de adjacência e representação CSR (*Compressed Sparse Row*). Os experimentos buscaram identificar o comportamento de cada estrutura em diferentes cenários de uso, observando tanto o tempo de execução das operações quanto o consumo estimado de memória.

Com base na análise teórica apresentada anteriormente, espera-se que cada representação apresente vantagens em cenários distintos. A representação por listas possui custo proporcional ao grau dos vértices e tende a apresentar bom desempenho em grafos esparsos e dinâmicos. A matriz de adjacência oferece consultas de existência de aresta em tempo constante, porém com elevado custo de memória. Já a representação CSR apresenta excelente compactação e boa eficiência para consultas em grafos estáticos, embora possua alto custo para inserções e remoções devido às conversões necessárias entre estruturas.

Dessa forma, a análise experimental busca validar empiricamente essas características e verificar como o desempenho varia em função do tamanho da rede social, da densidade do grafo e da frequência relativa entre operações de atualização e consulta.

5.1. Metodologia

Os experimentos foram executados utilizando entradas sintéticas geradas de forma controlada, permitindo variar isoladamente os principais parâmetros que afetam o custo das operações. Foram considerados dois perfis principais de carga:

- **Perfil com predominância de atualizações:** maior frequência de operações que modificam os grafos, como criação de usuários, adição e remoção de relações sociais e associação de temas;
- **Perfil com predominância de consultas:** maior frequência de operações de leitura, como consultas de seguidores, amigos, temas de interesse e verificação de relações.

Os parâmetros variados nos experimentos foram:

- número total de usuários;
- número total de temas;
- número médio de temas por usuário;
- densidade do grafo social.

Cada configuração experimental foi executada múltiplas vezes, sendo registrados os tempos médios de execução e o desvio padrão observado. Também foram estimados os custos de memória das diferentes representações. Os experimentos foram realizados sobre os três modos de armazenamento implementados.

5.2. Análise Geral do Tempo de Execução

A Figura 1 apresenta uma visão geral do comportamento das três representações de grafos nos diferentes perfis experimentais.

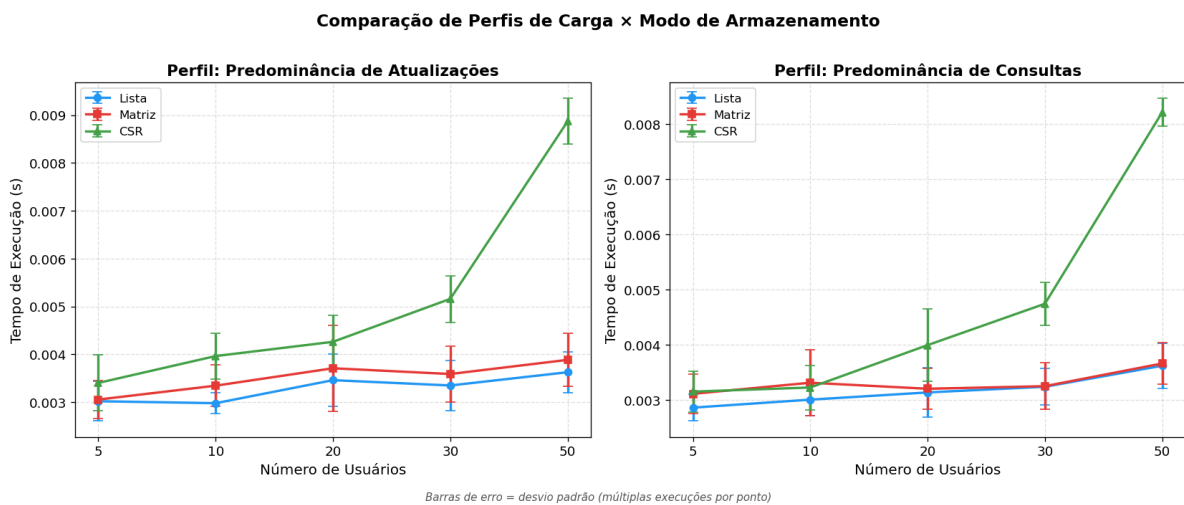


Figura 1: Comparação geral entre os perfis de atualizações e consultas

Os resultados obtidos mostram comportamento consistente entre todos os experimentos realizados. De maneira geral, as representações em lista de adjacência e matriz apresentaram desempenhos próximos na maior parte dos cenários avaliados, enquanto a representação CSR apresentou crescimento mais acentuado no tempo de execução conforme os grafos se tornavam maiores ou mais densos.

Nos cenários com predominância de atualizações, a lista de adjacência apresentou o melhor comportamento geral. Isso ocorre porque operações de inserção e remoção dependem principalmente do grau dos vértices, evitando reconstruções completas das estruturas internas. A matriz

de adjacência também apresentou desempenho competitivo, especialmente em instâncias menores, devido ao acesso direto aos elementos armazenados.

A representação CSR apresentou desempenho significativamente inferior nos cenários dinâmicos. Como inserções e remoções exigem conversões entre CSR e listas de adjacência, o custo dessas operações cresce rapidamente conforme aumenta o número de vértices e arestas do grafo. Esse comportamento se tornou mais evidente nos experimentos com maior número de usuários, maior densidade e maior quantidade de relações entre usuários e temas.

Nos cenários com predominância de consultas, a matriz de adjacência apresentou desempenho bastante competitivo, especialmente em operações de verificação de relações, devido ao acesso em tempo constante às posições da matriz. A lista de adjacência também manteve desempenho consistente, principalmente em grafos esparsos.

Embora a CSR apresente boa eficiência de armazenamento, seu desempenho experimental permaneceu inferior na maior parte dos cenários avaliados. Isso ocorreu principalmente devido ao caráter dinâmico do sistema, no qual operações de atualização são frequentes e acabam anulando as vantagens tradicionais dessa representação em grafos estáticos.

Os resultados experimentais também mostraram que o aumento do número de usuários, do número de temas, da densidade do grafo social e da quantidade média de temas por usuário afeta diretamente o desempenho das três representações. Em especial, o crescimento da densidade do grafo aumenta o custo das operações baseadas em percursos de vizinhança, impactando principalmente listas e CSR. Já a matriz sofre maior impacto no consumo de memória, embora mantenha custos de acesso mais estáveis.

De forma geral, os experimentos confirmam as expectativas levantadas pela análise assintótica apresentada anteriormente. Estruturas baseadas em listas apresentaram melhor equilíbrio entre custo computacional e flexibilidade para atualizações, enquanto a matriz se mostrou vantajosa para consultas rápidas em grafos mais densos. A CSR, apesar da excelente compactação de memória, mostrou-se menos adequada para cenários com modificações frequentes na estrutura do grafo.

5.3. Tempo Médio por Tipo de Operação

A Figura 2 apresenta a comparação do tempo médio das principais operações implementadas.

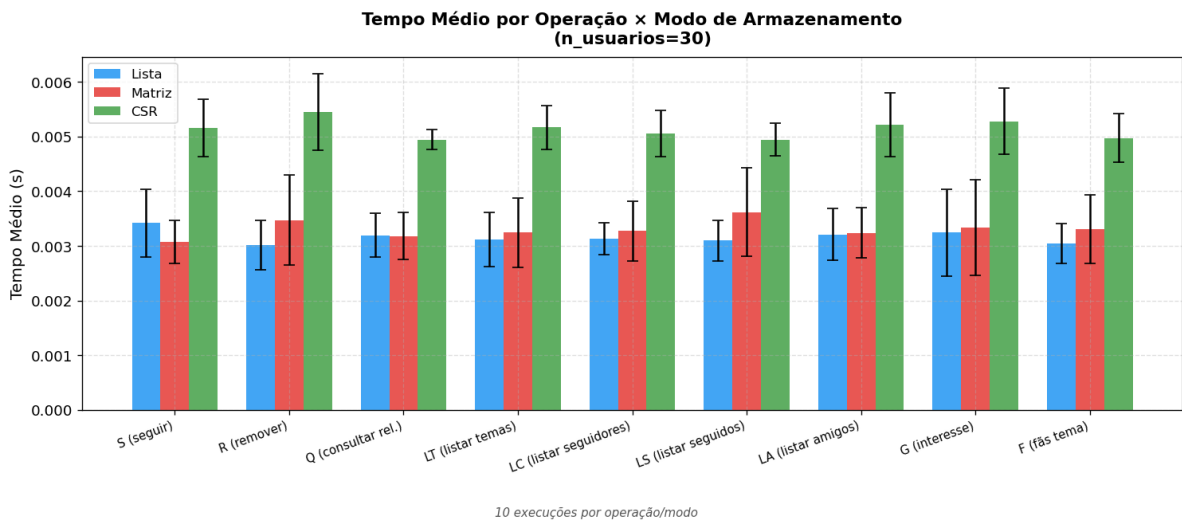


Figura 2: Tempo médio por tipo de operação

É possível notar como todas as operações apresentaram melhor desempenho nas representações em lista e matriz, o que evidencia que estas são as opções adequadas para realizar operações em

sistemas dinâmicos, em comparação com a CSR.

Nas operações de consulta envolvendo acessos únicos, a matriz frequentemente apresentou desempenho competitivo ou superior em razão do acesso direto às posições da estrutura, enquanto as listas são superiores em operações de recuperar todos os vizinhos.

5.4. Uso de Memória

A Figura 3 apresenta a estimativa de uso de memória para cada representação em função do número de usuários.

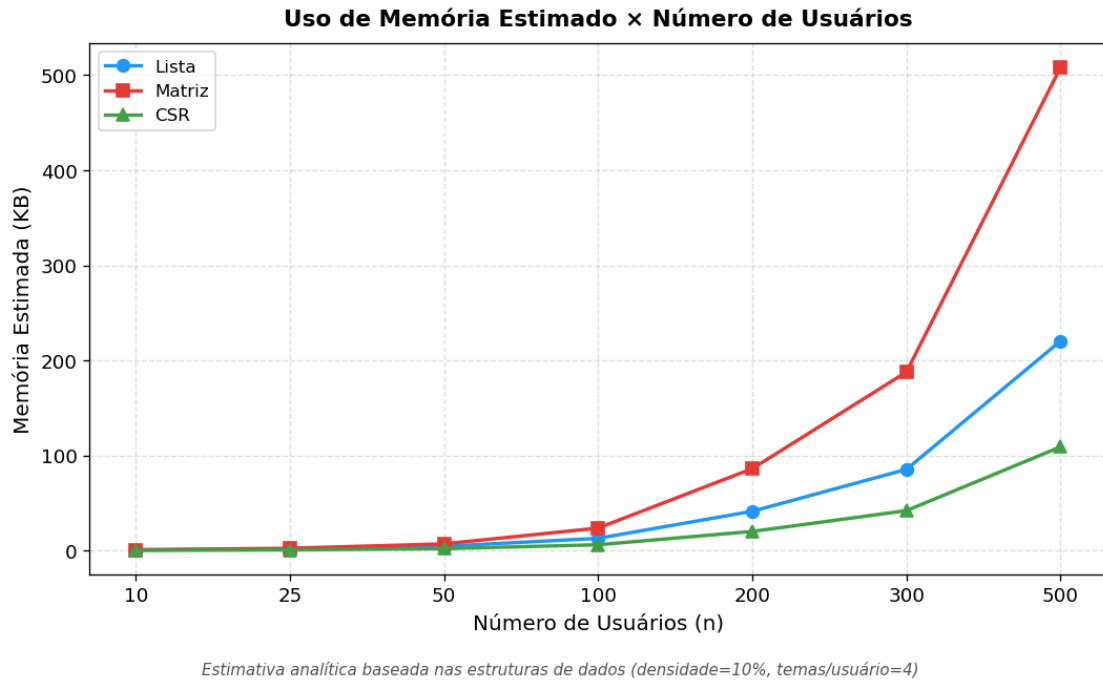


Figura 3: Uso estimado de memória

Os resultados confirmam o comportamento esperado teoricamente. A matriz de adjacência apresentou crescimento quadrático em relação ao número de usuários, tornando-se rapidamente a representação com maior consumo de memória.

A lista de adjacência apresentou crescimento aproximadamente proporcional ao número de arestas, mantendo consumo significativamente inferior ao da matriz em grafos esparsos.

A representação CSR foi a mais econômica em memória, devido ao armazenamento compacto dos índices e arestas em vetores contíguos, eliminando ponteiros extras presentes nas listas.

5.5. Discussão dos Resultados

Os resultados experimentais confirmam as expectativas levantadas pela análise teórica de complexidade.

A lista de adjacência apresentou o melhor equilíbrio geral entre desempenho e uso de memória, especialmente em cenários dinâmicos e grafos esparsos. Como o custo das operações depende principalmente do grau dos vértices, essa representação se mostrou eficiente para inserções, remoções e consultas locais.

A matriz de adjacência apresentou desempenho competitivo em operações de consulta de existência de arestas, principalmente em grafos mais densos. Entretanto, seu elevado custo de memória limita sua escalabilidade para instâncias maiores.

A representação CSR demonstrou excelente eficiência de armazenamento, mas desempenho inferior em cenários dinâmicos. O alto custo associado às conversões necessárias para atualizações tornou essa representação inadequada para cargas com grande frequência de modificações. Mas é a opção ideal para se armazenar o estado do sistema estaticamente, como em um backup.

De forma geral, os experimentos mostram que não existe uma representação universalmente superior. A escolha da estrutura mais adequada depende diretamente das características do cenário de uso, especialmente da densidade do grafo e da frequência relativa entre consultas e atualizações.

6. Conclusão

Este trabalho apresentou o desenvolvimento de um sistema para modelagem de redes sociais acadêmicas na plataforma ConectaDCC, utilizando diferentes representações de grafos para armazenar relações sociais e interesses entre usuários e temas.

Ao longo do desenvolvimento, foram exploradas estruturas de dados baseadas em listas de adjacência, matrizes de adjacência e CSR (*Compressed Sparse Row*), além da utilização de Tipos Abstratos de Dados para modularizar os componentes do sistema. A análise de complexidade permitiu compreender como o custo das operações varia de acordo com a representação escolhida, evidenciando os impactos da densidade do grafo, do número de vértices e da frequência de consultas e atualizações.

A partir da análise experimental, foi possível observar na prática o comportamento das diferentes representações em cenários variados de utilização. Os resultados mostraram que listas de adjacência oferecem o melhor equilíbrio geral entre desempenho e consumo de memória em grafos dinâmicos e esparsos. A matriz de adjacência apresentou vantagens em operações de consulta, especialmente em grafos mais densos, enquanto a CSR demonstrou elevada eficiência de armazenamento, mas desempenho inferior em cenários com modificações frequentes devido ao custo das reconstruções internas.

De forma geral, o trabalho permitiu consolidar conceitos fundamentais relacionados a estruturas de dados, representação de grafos, análise de algoritmos e avaliação experimental de desempenho. Além disso, evidenciou a importância de escolher adequadamente as estruturas utilizadas de acordo com as características do problema e do perfil de utilização do sistema.

Referências

1. Anisio Lacerda, Giesele L. Pappa, Marcio Santos e Wagner Meira Jr. (2026). *Especificação do Trabalho Prático 2: Modelagem de Redes Sociais Acadêmicas*. Disponibilizado via Moodle. Departamento de Ciência da Computação, UFMG.
2. Anisio Lacerda, Giesele L. Pappa, Marcio Santos e Wagner Meira Jr. (2026). *Slides virtuais da disciplina de Estruturas de Dados*. Disponibilizado via Moodle. Departamento de Ciência da Computação, UFMG.
3. SAAD, Yousef. *Iterative methods for sparse linear systems*. 2. ed. Philadelphia: Society for Industrial and Applied Mathematics, 2003.
4. BULUÇ, Aydin; GILBERT, John R. *The combinatorial BLAS: design, implementation, and applications*. International Journal of High Performance Computing Applications, v. 25, n. 4, p. 496–509, 2011.